



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Processamento de Linguagens

Ano Letivo de 2025/2026

Construção de um Compilador para Fortran 77 Standard

Carlos Araújo

A106845

Vitor Senra

A106921

Hugo Soares

A107293

May 16, 2026

PL

Índice

1. Introdução	1
2. Arquitetura do Compilador	1
3. Análise Léxica	1
4. Análise Sintática	2
5. Análise Semântica e Tabela de Símbolos	2
6. Representação Intermédia (AST)	3
7. Otimizações e Geração de Código	3
7.1. Otimização (Middle-end)	3
7.2. Geração de Código VM (Back-end)	4
7.2.1. Mapeamento de Construções Fortran	4
7.2.2. Subprogramas (FUNCTION / SUBROUTINE)	4
8. Testes e Validação	5
9. Conclusões	5

1. Introdução

A disciplina de Processamento de Linguagens foca no estudo de compiladores e estruturas interpretativas. Este projeto consubstancia estes conhecimentos e implementa um compilador integral para a norma original clássica **Fortran 77**, que tem como base a geração estrita de instruções orientadas a uma Stack Virtual Machine (fornecida no âmbito curricular). O programa foi desenhado em **Python 3** recorrendo ativamente à biblioteca de *parsing* **PLY**.

O compilador segue e adota na íntegra a arquitetura standard de *compilers*, implementando um fluxo pipeline rigoroso modularizado em *Front-end* e *Back-end*: pré-processamento de fixed-formats, análise léxica para streams de tokens sintáticos, construção de Árvore AST, análise semântica acoplada por escopo em *Symbol Tables* e geração nativa de código .vm e otimizações. O presente relatório técnico descreve em detalhe os padrões, fluxos, restrições e as abordagens de *engineering* seguidas no projeto ao longo das sucessivas camadas.

2. Arquitetura do Compilador

O compilador possui módulos independentes em `src/`:

- `lexer.py`: Tokenização (via PLY).
- `parser.py`: Análise sintática e construção da AST.
- `ast_nodes.py`: Nós da árvore abstrata usando `@dataclass`.
- `symbol_table.py`: Tabela de símbolos com escopo.
- `optimizer.py`: Otimizações na AST (Constant Folding, Dead Code Elimination).
- `codegen.py`: Geração de instruções VM (Stack machine).

O `main.py` orquestra o pipeline completo: lê .for, gera tokens, constrói a AST, otimiza a árvore, e emite código para um ficheiro .vm. Aceita o *flag* opcional `--comments` (ou `-c`) para incluir comentários explicativos no ficheiro .vm gerado (por omissão, o output é limpo). A separação em fases (Front-end, Middle-end/Optimizer, Back-end) garante uma arquitetura modular, fácil de testar (via `pytest`) e manter.

3. Análise Léxica

A análise léxica é o primeiro passo da conduta, garantindo a decomposição da fonte Fortran num fluxo contínuo de *tokens* validados. Desenvolvida no módulo `lexer.py` usando `ply.lex`, garante o tratamento nativo do **Fixed-Format** do Fortran 77:

- **Colunas 1-5**: Reservadas unicamente para numeração de identificação local de instruções (Labels/GOTO targets).
- **Coluna 6**: Reservada como caracter de controlo de contiguidade de linha ou denotação de comentário (quando C ou *).
- **Colunas 7-72**: Espaço principal para a escrita do corpo sintático.

A nível preparatório, o pré-processador interno normaliza o source *in memory* para remover os comentários explícitos e unir quebras formatadas contíguas.

Os Tokens emitidos para o Parser incluem as *keywords* base da norma (PROGRAM, IF, DO, CONTINUE), operadores lógicos nativos e transacionais (.GT., .AND., **), literais de múltiplas tipagens (Inteiros, Reais com formato E, literais String entre aspas isoladas, ou lógicos como .TRUE.). Como estipulado, todos os identificadores (tendo tamanho limite de 6 caracteres alfanuméricos começados por letra) são recolhidos e sistematicamente transformados em letras maiúsculas (.upper()), respeitando a **Case-Insensitivity** fundamental do Fortran 77. Em caso de divergência ortográfica na formação, o lexer atira uma LexerError reportando a posição.

4. Análise Sintática

A análise sintática baseou-se na implementação rigorosa da Context-Free Grammar (CFG) desenvolvida para a norma em questão (ver anexo em formato CFG.md). A sua base é LALR(1) suportada diretamente pelas implementações estritas da engine PLY Yacc via módulo parser.py.

A priorização operacional de expressões avaliadas obedece estritamente às tabelas da linguagem: primeiramente operações lógicas, relacionais, até à base aritmética (logical_or → logical_and → relational → additive → multiplicative → power → unary → primary).

Concomitantemente à validação gramatical da cadeia, o módulo sintático invoca sincronamente a infraestrutura Semântica, abortando imediatamente se as regras de alocação forem violadas (variáveis declaradas, acessos a arrays unidimensionais, blocos aninhados e validação de destinos GOTO). No final da verificação descendente com sucesso em cada regra gramatical (p_[rule]), instanciam-se *Dataclasses* para a construção da **Abstract Syntax Tree (AST)**. Em caso de divergência estrita (falta de parêntesis, terminação incorreta de loop DO, etc.), é lançada uma ParserSyntaxError capturada em main.py com mensagem legível.

5. Análise Semântica e Tabela de Símbolos

Durante a passadeira sintática LALR(1), o compilador invoca sincronamente e *inline* os métodos expostos na sua **Tabela de Símbolos**. O symbol_table.py materializa-se como uma pilha LIFO suportando Nested Scopes (escopos aninhados), com o global como raiz partilhada. Cada símbolo é representado por uma Dataclass Symbol que preserva a metainformação essencial: nome, tipo (INTEGER, REAL, LOGICAL, CHARACTER), espacialidade (escalar ou array unidimensional), linha de declaração, e flag de inicialização.

A validação de conflitos (lançada como SemanticError) ocorre perante as seguintes regras:

- Pré-declaração obrigatória: todas as variáveis, arrays e parâmetros devem ser declarados antes do uso (lookup()).

- Sem re-declarações (`declare()`): sombras ou duplicados na mesma *frame* são rejeitados.
- Parâmetros (`PARAMETER`): constantes imutáveis declaradas via `PARAMETER`. O compilador rejeita duplicados, atribuições a parâmetros, e aplicações de `PARAMETER` a arrays.
- Acesso a arrays: verificação de dimensionalidade (`check_array_access()`) — o subscrito deve ser único e, quando possível, os índices literais são validados em tempo de compilação contra os limites declarados.
- Compatibilidade de tipos: atribuições `REAL` → `INTEGER` admitem truncação implícita; `LOGICAL` → numérico (ou vice-versa) é rejeitado.

6. Representação Intermédia (AST)

A estruturação final das validações recai na geração da **Abstract Syntax Tree (AST)**. Em oposição às implementações baseadas puramente em tuplos nativos em Python, adotou-se o uso estrito de módulos `@dataclass` (`ast_nodes.py`), que propiciam uma semântica rígida, documentada e com verificação de tipo.

A infraestrutura ramifica-se nos seguintes nós de topo:

- `CompilationUnit` e `Program`: O ponto fulcral e originário da representação para o main script e subprogramas.
- *Declarações*: Suportadas via `TypeDeclaration` e parâmetros globais imutáveis.
- *Statements (Instruções)*: Enquadramentos complexos de fluxo via `IfThenElse` (com suporte a ramificações múltiplas de *elseif*), `DoLoop`, saltos explícitos via `GotoStatement`, e as primitivas IO `ReadStatement` / `PrintStatement`.
- *Expressões (Expressions)*: Cálculos puramente avaliados via `BinaryOp`, `UnaryOp`, e resoluções terminadas em `Literal` ou chamadas recursivas `FunctionCall`.

Todos os nós estendem rigorosamente uma classe base abstracta genérica `Node`, na qual o rastreamento da linha (`lineno`) garante depuração clara caso a *Pipeline* intercetar falhas na Geração ou Otimização. Em última análise, a AST limpa e validada garante o isolamento funcional do Front-end em relação ao Back-end.

7. Otimizações e Geração de Código

7.1. Otimização (Middle-end)

Antes de gerar código, a Árvore Sintática Abstrata (AST) passa por um módulo de otimização (`optimizer.py`). Implementámos passagens de otimização focadas na melhoria de performance (valorização), nomeadamente:

- **Constant Folding**: Expressões binárias (+, -, *, /, **) e operadores relacionais/lógicos entre literais são avaliados em tempo de compilação, incluindo os limites de ciclos D0.

- **Dead Code Elimination (DCE):** Remoção de código inatingível (instruções após GOTO / RETURN sem label) e eliminação de **dead stores** (atribuições cujo valor nunca é lido posteriormente no mesmo bloco).
- **IF Constant Folding:** Quando a condição de um IF é um literal lógico (. TRUE. ou . FALSE.), o ramo correspondente é colapsado diretamente.

7.2. Geração de Código VM (Back-end)

O módulo `codegen.py` implementa o padrão **Visitor** para percorrer a AST e gerar um ficheiro `.vm` com as instruções para a máquina virtual baseada em pilha (stack machine).

7.2.1. Mapeamento de Construções Fortran

- **Variáveis e Atribuições:** Utiliza-se um contador global de endereços. A escrita faz-se com `STOREG addr` (globais) ou `STOREL addr` (locais em subprogramas) e a leitura com `PUSHG / PUSHL`. Arrays unidimensionais alocam-se no *heap* estruturado via `ALLOC` e acedem-se com `LOADN / STOREN` (indexação convertida de 1-based para 0-based).
- **Tipos Numéricos:** O compilador distingue `INTEGER` e `REAL`. Operações aritméticas com pelo menos um operando `REAL` geram opcodes float (`FADD`, `FSUB`, `FMUL`, `FDIV`, `FINF`, `FINFEQ`, etc.) e inserem conversões automáticas (`ITOF`, `FTOI`). I/O de reais utiliza `WRITEF / ATOF`.
- **Aritmética e Lógica:** Expressões são avaliadas em pós-ordem. Operadores inteiros geram `ADD`, `SUB`, `MUL`, `DIV`, `INFEQ`, `SUPEQ`, etc.
- **Controlo de Fluxo (IF / GOTO):** As *labels* de Fortran são mapeadas para *labels* VM unívocas. Instruções como `JZ` (Jump if Zero) e `JUMP` realizam os desvios.
- **Ciclos (DO):** Um `DO` loop inicializa a variável de iteração, testa a condição (`INFEQ`), corre o corpo, incrementa, e repete via `JUMP`.
- **I/O:** `READ` extrai *strings* do input e converte via `ATOI / ATOF` antes de armazenar. `PRINT` coloca as expressões na pilha e emite `WRITEI`, `WRITEF`, `WRITES` ou `WRITELN`.

7.2.2. Subprogramas (FUNCTION / SUBROUTINE)

A convenção de chamada segue o modelo da VM: o *caller* empilha o *slot* de retorno, os argumentos por ordem inversa, e o endereço da função (`PUSHA + CALL`). No *callee*, os parâmetros acedem-se com offsets negativos de `PUSHL` (ex: `PUSHL -1`, `PUSHL -2`) e as variáveis locais alocam-se com `PUSHN`. Antes de cada `RETURN`, o compilador emite `POP n` para libertar o espaço de pilha ocupado pelas locais, garantindo que o `sp` é restaurado corretamente. A potenciação (`**`) e o operador `MOD` são suportados via *helpers* runtime (`POW:` e `MOD:`) anexados no final do ficheiro `.vm` apenas quando necessário.

8. Testes e Validação

O compilador foi validado de ponta a ponta com cinco programas baseados nos cenários de teste da disciplina. Para automatizar este processo, foi desenvolvida uma *test suite* em Python utilizando a biblioteca `pytest` (`tests/test_compiler.py`).

A automação garante a verificação contínua (CI) e testa os seguintes programas:

- `hello.for`: Avalia rotinas básicas de I/O (PRINT de *strings*).
- `fatorial.for`: Valida o cálculo matemático, controlo de fluxo complexo iterativo (DO) e resolução de variáveis escalares inteiras.
- `primo.for`: Verifica ciclos, manipulação lógica e construções condicionais (IF/GOTO) e ramificações condicionais baseadas em booleanos implícitos.
- `somaarr.for`: Testa o correto parseamento de dimensões, declarações e acesso por indexação a Arrays Unidimensionais (1D).
- `conversor.for`: Valida a compatibilidade e a transição entre múltiplos subprogramas (Funções e Procedimentos).

Em todos os casos, a suite não só avalia o sucesso da fase léxica e sintática (sem levantar erros do parser/lexer) mas constrói a AST, realiza as otimizações Middle-end via `optimizer.py`, e efetivamente emite o binário (`.vm`). Todos os 5 testes garantem retorno de código de erro zero.

9. Conclusões

Este projeto demonstrou, com sucesso, a implementação rigorosa de um compilador de uma linguagem rica, como Fortran 77, em Python. Cumpriram-se os requisitos propostos com distinção. A modularidade (via Visitor e abordagens pipeline) facilitou a manutenção e estensibilidade.

As principais dificuldades envolveram o tratamento estrito de *labels* via formatação fixa clássica e o mapeamento fiel em arquitetura stack-machine. Outro aspeto desafiante consistiu em garantir que nós AST não-tradicionais fossem resolvidos e alocados uniformemente sem dependências cruzadas (resolvido por resolução de “post-order”).

O trabalho supera a cota de 10/20 ao realizar alocação com a Tabela de Símbolos, passagens bónus como Constant Folding e DCE via `optimizer.py`, garantindo performance para a VM. Como trabalho futuro propomos a integração de Common Subexpression Elimination através da geração local de um Control Flow Graph. Em suma, desenvolveu-se uma ferramenta robusta e academicamente completa.